



APPLIED DATA SCIENCE &
ARTIFICIAL INTELLIGENCE



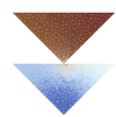
UNIVERSITÀ
DEGLI STUDI
DI TRIESTE

Bridging Logic and Learning: Decoding Temporal Logic Embeddings via Transformers

Sara Candussio¹ Gaia Saveri¹ Gabriele Sarti² Luca Bortolussi¹

¹AILab, MIGe, University of Trieste

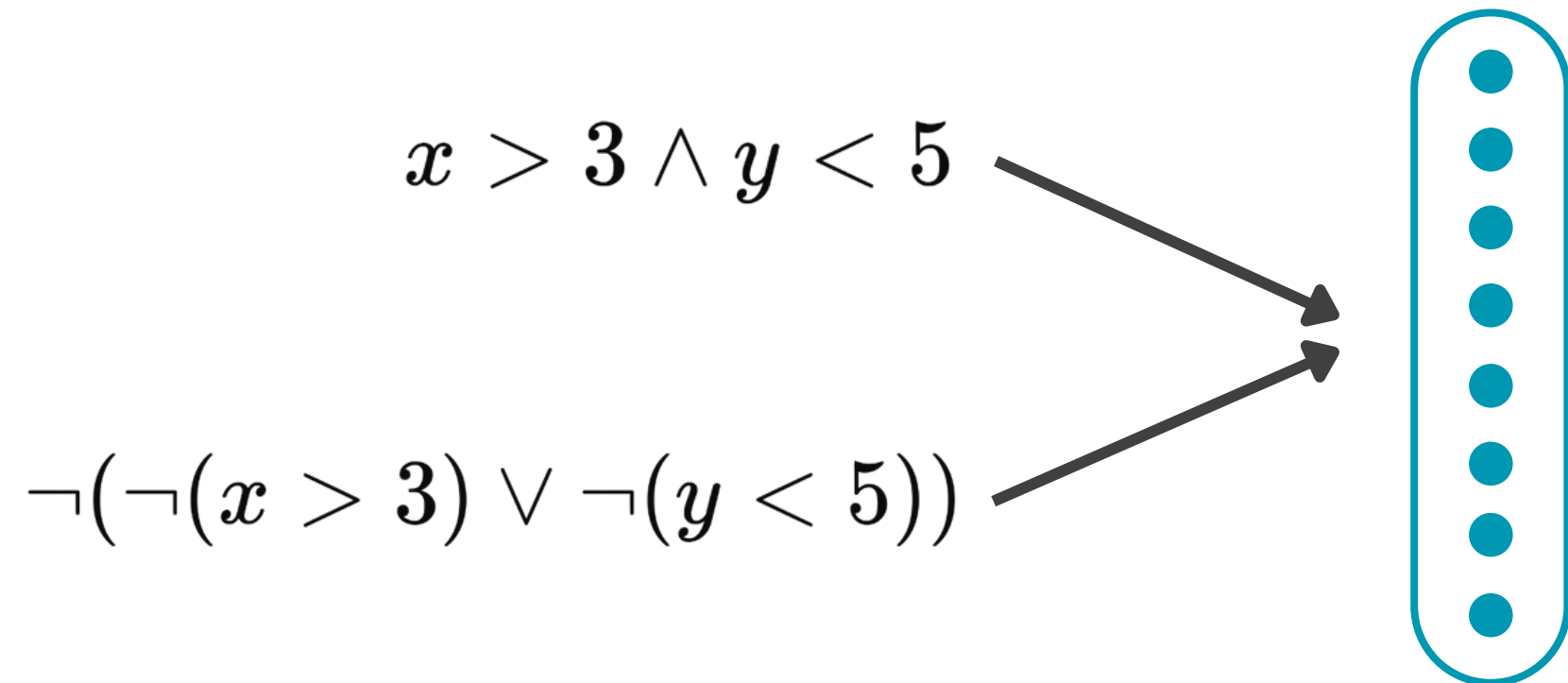
²Center for Language and Cognition (CLCG), University of Groningen

ECML 
◆ PKDD
2025 

Kernel-based embedding

- formula to embed φ
 - anchor set of formulae $\{\psi_1, \dots, \psi_d\}$
 - kernel/similarity function $k(\varphi, \cdot) \in \mathbb{R}^d$
- $$\left. \begin{array}{l} \bullet \text{ formula to embed } \varphi \\ \bullet \text{ anchor set of formulae } \{\psi_1, \dots, \psi_d\} \\ \bullet \text{ kernel/similarity function } k(\varphi, \cdot) \in \mathbb{R}^d \end{array} \right\} e_\varphi = [k(\varphi, \psi_1), \dots, k(\varphi, \psi_d)] \in \mathbb{R}^d$$

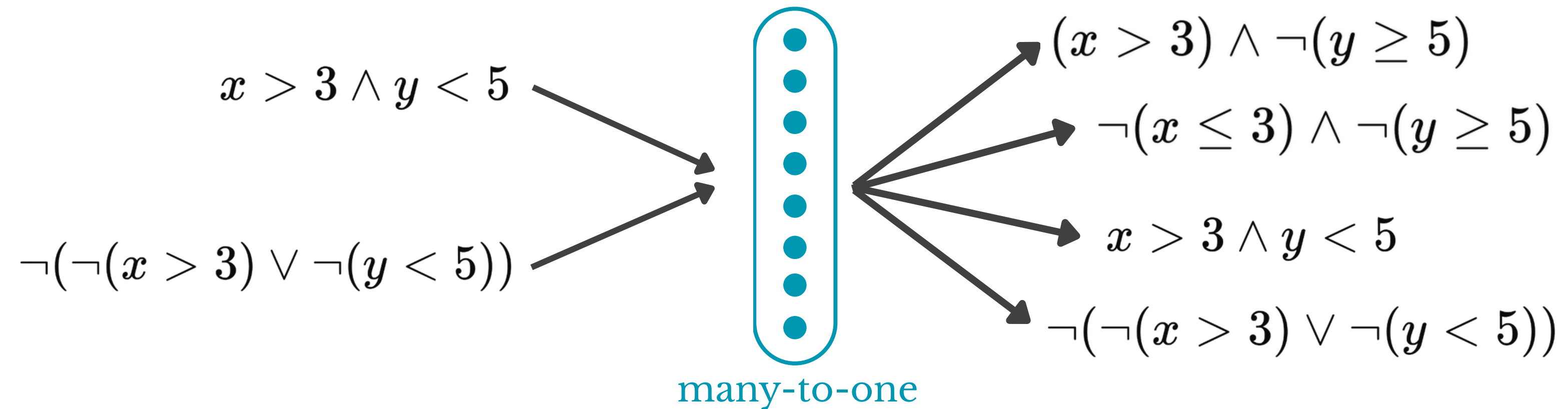
✓ map objects into semantics-preserving continuous representations



Kernel-based embedding

- formula to embed φ
 - anchor set of formulae $\{\psi_1, \dots, \psi_d\}$
 - kernel/similarity function $k(\varphi, \cdot) \in \mathbb{R}^d$
- $e_\varphi = [k(\varphi, \psi_1), \dots, k(\varphi, \psi_d)] \in \mathbb{R}^d$

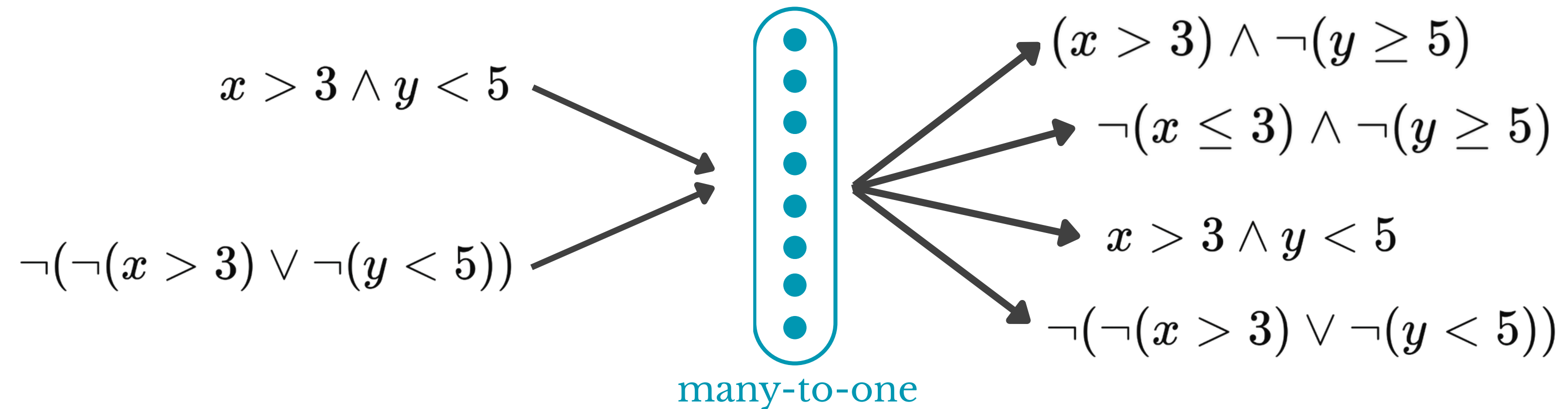
✓ map objects into semantics-preserving continuous representations



Kernel-based embedding

- formula to embed φ
 - anchor set of formulae $\{\psi_1, \dots, \psi_d\}$
 - kernel/similarity function $k(\varphi, \cdot) \in \mathbb{R}^d$
- $e_\varphi = [k(\varphi, \psi_1), \dots, k(\varphi, \psi_d)] \in \mathbb{R}^d$

✓ map objects into semantics-preserving continuous representations



✗ are generally not invertible

Signal Temporal Logic (STL)

is a *linear-time temporal logic* which expresses **properties on trajectories over dense time intervals**.

Signals (or trajectories) are here defined as functions $\xi : I \rightarrow D$ over the time domain mapping in the state space.

The **syntax** of STL is given by:

$$\varphi := tt \mid \pi \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \mathbf{U}_{[a,b]} \varphi_2$$

From $\neg$ and $\wedge$, it is possible to derive also the $\vee$; from $\mathbf{U}[a,b]$, the *until* operator, the *eventually* $\mathbf{F}[a,b]$ and the *always* $\mathbf{G}[a,b]$ operators.

Signal Temporal Logic (STL)

is a *linear-time temporal logic* which expresses **properties on trajectories over dense time intervals**.

Signals (or trajectories) are here defined as functions $\xi : I \rightarrow D$ over the time domain mapping in the state space.

The syntax of STL is given by:

$$\varphi := tt \mid \pi \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \mathbf{U}_{[a,b]} \varphi_2$$

From $\neg$ and $\wedge$, it is possible to derive also the $\vee$; from $\mathbf{U}[a,b]$, the *until* operator, the *eventually* $\mathbf{F}[a,b]$ and the *always* $\mathbf{G}[a,b]$ operators.

$\mathbf{F}_{[0,10]}(\text{temperature} > 25)$

within the next 10 minutes, the temperature will exceed 25°C

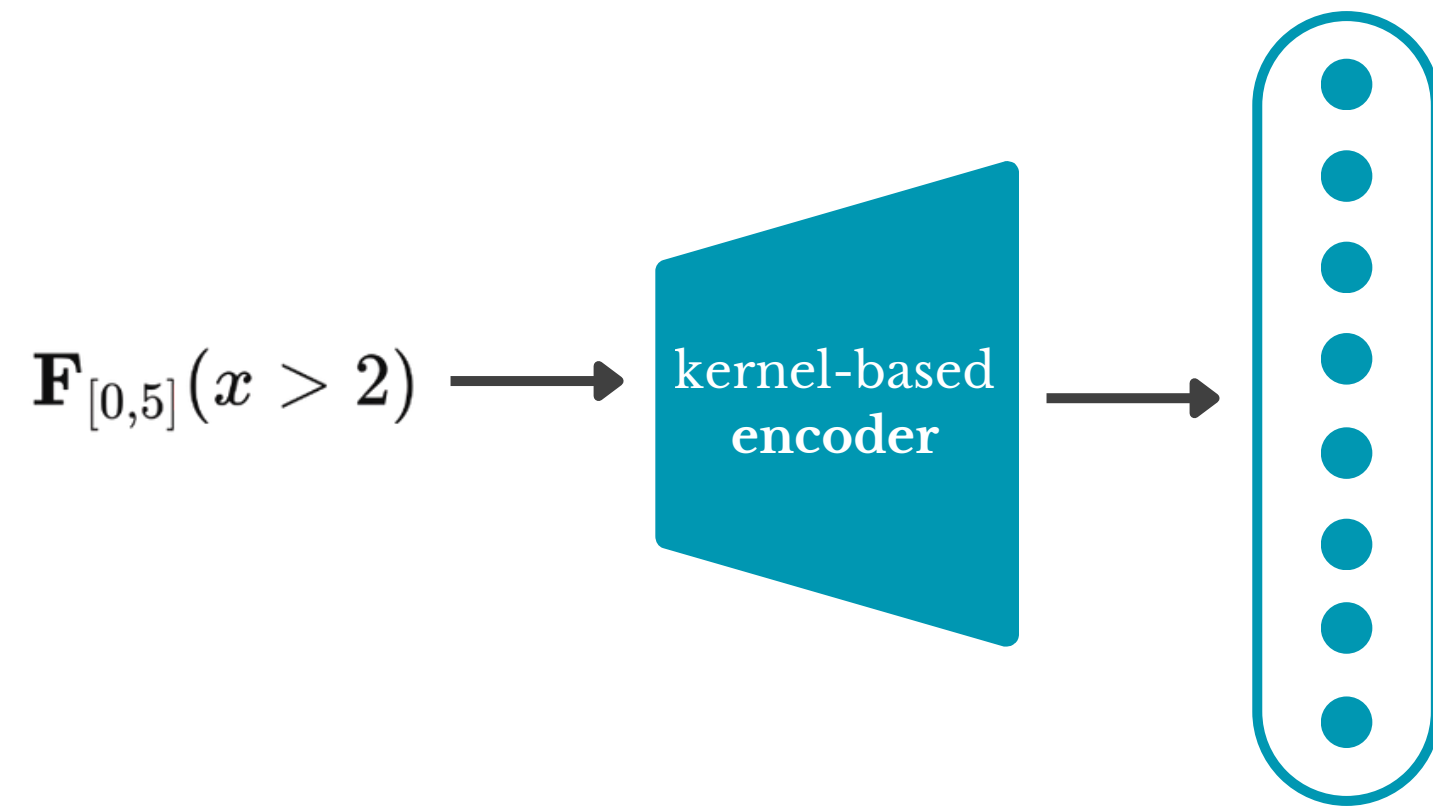
Decoding formulae

We can treat STL syntax elements **as tokens of a language**, allowing a **decoder** to *translate* a semantic-preserving embedding back into a formula.

Decoding formulae

We can treat STL syntax elements as **tokens of a language**, allowing a **decoder** to *translate* a semantic-preserving embedding back into a formula.

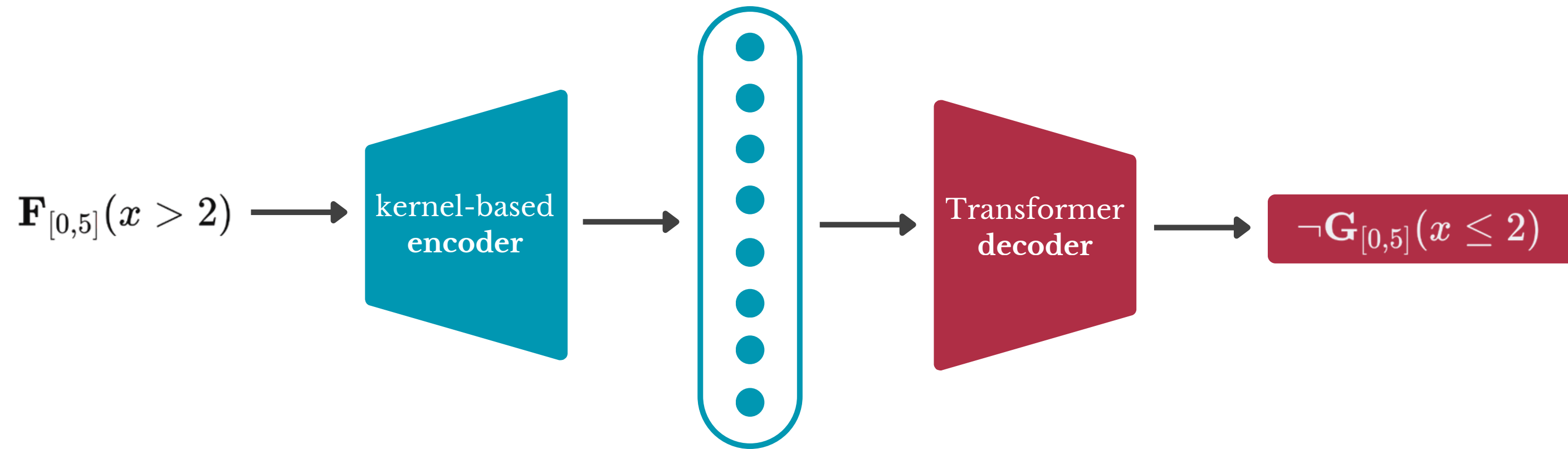
(1) pass the formula through the kernel-based encoder and **obtain the embedding**



Decoding formulae

We can treat STL syntax elements as **tokens of a language**, allowing a **decoder** to *translate* a semantic-preserving embedding back into a formula.

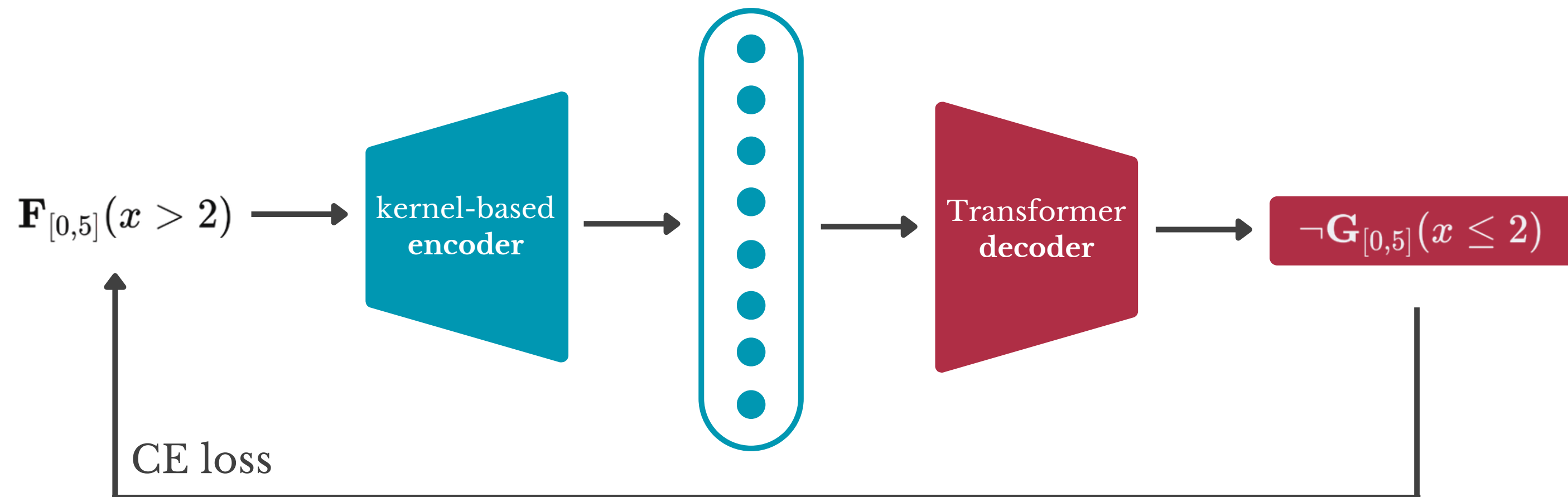
(2) **decode** that semantic-preserving continuous **embedding** into a formula



Decoding formulae

We can treat STL syntax elements as **tokens of a language**, allowing a **decoder** to *translate* a semantic-preserving embedding back into a formula.

(3) train the decoder to reconstruct the formula by Cross-Entropy loss

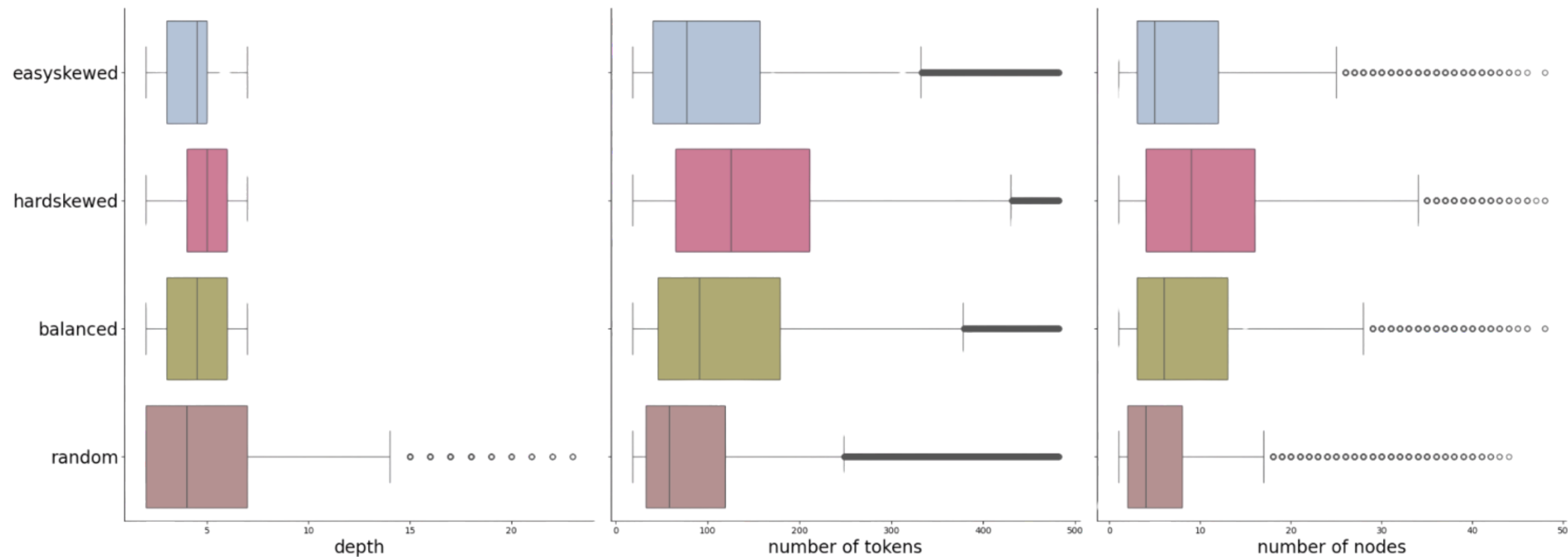


⚠ *Cross-Entropy loss favours exact matches* ⚠

Training data

☞ determine whether the model **truly learns semantics**, *not just patterns from a specific distribution*

🔍 We crafted 4 training sets, each with 78k formulae with different complexities:



Results

- 🔧 12 layers, 16 heads each, 4096 FFN dimension
2 embedding sizes: **512** (*small*) and **1024**,
24k (10 epochs) and **48k** (20 epochs) training steps
- 📏 euclidean distance, cosine similarity, difference in satisfiability

Results

🔧 12 layers, 16 heads each, 4096 FFN dimension

2 embedding sizes: **512** (*small*) and **1024**,

24k (10 epochs) and **48k** (20 epochs) training steps

📏 euclidean distance, cosine similarity, difference in satisfiability

Model	$d(\boldsymbol{\rho}(\varphi), \boldsymbol{\rho}(\hat{\varphi}))$				$\cos(\boldsymbol{\rho}(\varphi), \boldsymbol{\rho}(\hat{\varphi}))$			
	1quart	median	3quart	99perc	1quart	median	3quart	99perc
DB ←	16.51 (1)	28.13 (4)	43.87 (12)	81.46 (75)	0.795 (91)	0.9262 (98)	0.9802 (99)	0.9995 (100)
random (step 24K)	8.447 (1)	17.63 (2)	36.03 (7)	123.4 (100)	0.8617 (95)	0.9718 (99)	0.9939 (100)	1.000 (100)
random small (step 24K)	11.66 (1)	23.06 (2)	44.51 (12)	113.4 (100)	0.7985 (91)	0.9518 (99)	0.9885 (100)	1.000 (100)
random (step 48K)	6.054 (0)	12.71 (1)	29.89 (4)	110.1 (100)	0.9012 (96)	0.9854 (100)	0.9971 (100)	1.000 (100)
random small (step 48K)	7.462 (1)	16.01 (1)	31.22 (5)	101.1 (100)	0.8944 (96)	0.9774 (99)	0.9952 (100)	1.000 (100)

$\text{diff}(\boldsymbol{s}(\varphi), \boldsymbol{s}(\hat{\varphi}))$			
1quart	median	3quart	99perc
0.0225 (1)	0.0908 (4)	0.1797 (11)	0.4831 (47)
0.0159 (1)	0.0423 (2)	0.1354 (7)	0.7571 (85)
0.0224 (1)	0.0586 (3)	0.1631 (9)	0.7154 (81)
0.0073 (0)	0.0274 (1)	0.1085 (6)	0.6625 (75)
0.0106 (1)	0.0359 (2)	0.1159 (6)	0.5563 (61)

Results

🔧 12 layers, 16 heads each, 4096 FFN dimension

2 embedding sizes: **512** (*small*) and **1024**,

24k (10 epochs) and **48k** (20 epochs) training steps

📏 euclidean distance, cosine similarity, difference in satisfiability

Model	$d(\boldsymbol{\rho}(\varphi), \boldsymbol{\rho}(\hat{\varphi}))$				$\cos(\boldsymbol{\rho}(\varphi), \boldsymbol{\rho}(\hat{\varphi}))$			
	1quart	median	3quart	99perc	1quart	median	3quart	99perc
DB	16.51 (1)	28.13 (4)	43.87 (12)	81.46 (75)	0.795 (91)	0.9262 (98)	0.9802 (99)	0.9995 (100)
random (step 24K)	8.447 (1)	17.63 (2)	36.03 (7)	123.4 (100)	0.8617 (95)	0.9718 (99)	0.9939 (100)	1.000 (100)
random small (step 24K)	11.66 (1)	23.06 (2)	44.51 (12)	113.4 (100)	0.7985 (91)	0.9518 (99)	0.9885 (100)	1.000 (100)
random (step 48K)	6.054 (0)	12.71 (1)	29.89 (4)	110.1 (100)	0.9012 (96)	0.9854 (100)	0.9971 (100)	1.000 (100)
random small (step 48K)	7.462 (1)	16.01 (1)	31.22 (5)	101.1 (100)	0.8944 (96)	0.9774 (99)	0.9952 (100)	1.000 (100)

$\text{diff}(\boldsymbol{s}(\varphi), \boldsymbol{s}(\hat{\varphi}))$				
1quart	median	3quart	99perc	
0.0225 (1)	0.0908 (4)	0.1797 (11)	0.4831 (47)	
0.0159 (1)	0.0423 (2)	0.1354 (7)	0.7571 (85)	
0.0224 (1)	0.0586 (3)	0.1631 (9)	0.7154 (81)	
0.0073 (0)	0.0274 (1)	0.1085 (6)	0.6625 (75)	
0.0106 (1)	0.0359 (2)	0.1159 (6)	0.5563 (61)	

Results

🔧 12 layers, 16 heads each, 4096 FFN dimension

2 embedding sizes: **512** (*small*) and **1024**,

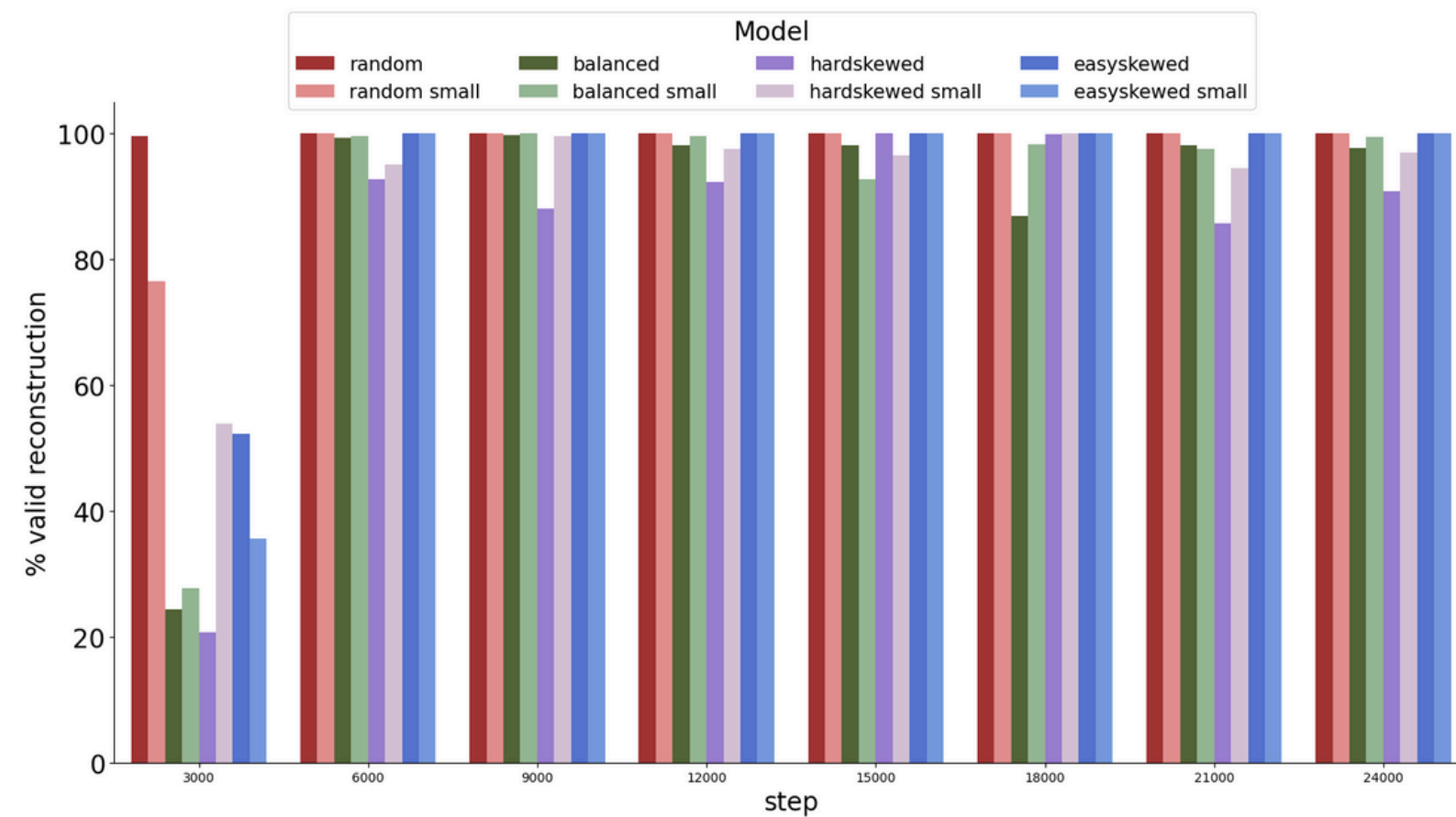
24k (10 epochs) and **48k** (20 epochs) training steps

📏 euclidean distance, cosine similarity, difference in satisfiability

Model	$d(\rho(\varphi), \rho(\hat{\varphi}))$				$\cos(\rho(\varphi), \rho(\hat{\varphi}))$			
	1quart	median	3quart	99perc	1quart	median	3quart	99perc
DB	16.51 (1)	28.13 (4)	43.87 (12)	81.46 (75)	0.795 (91)	0.9262 (98)	0.9802 (99)	0.9995 (100)
random (step 24K)	8.447 (1)	17.63 (2)	36.03 (7)	123.4 (100)	0.8617 (95)	0.9718 (99)	0.9939 (100)	1.000 (100)
random small (step 24K)	11.66 (1)	23.06 (2)	44.51 (12)	113.4 (100)	0.7985 (91)	0.9518 (99)	0.9885 (100)	1.000 (100)
random (step 48K)	6.054 (0)	12.71 (1)	29.89 (4)	110.1 (100)	0.9012 (96)	0.9854 (100)	0.9971 (100)	1.000 (100)
random small (step 48K)	7.462 (1)	16.01 (1)	31.22 (5)	101.1 (100)	0.8944 (96)	0.9774 (99)	0.9952 (100)	1.000 (100)

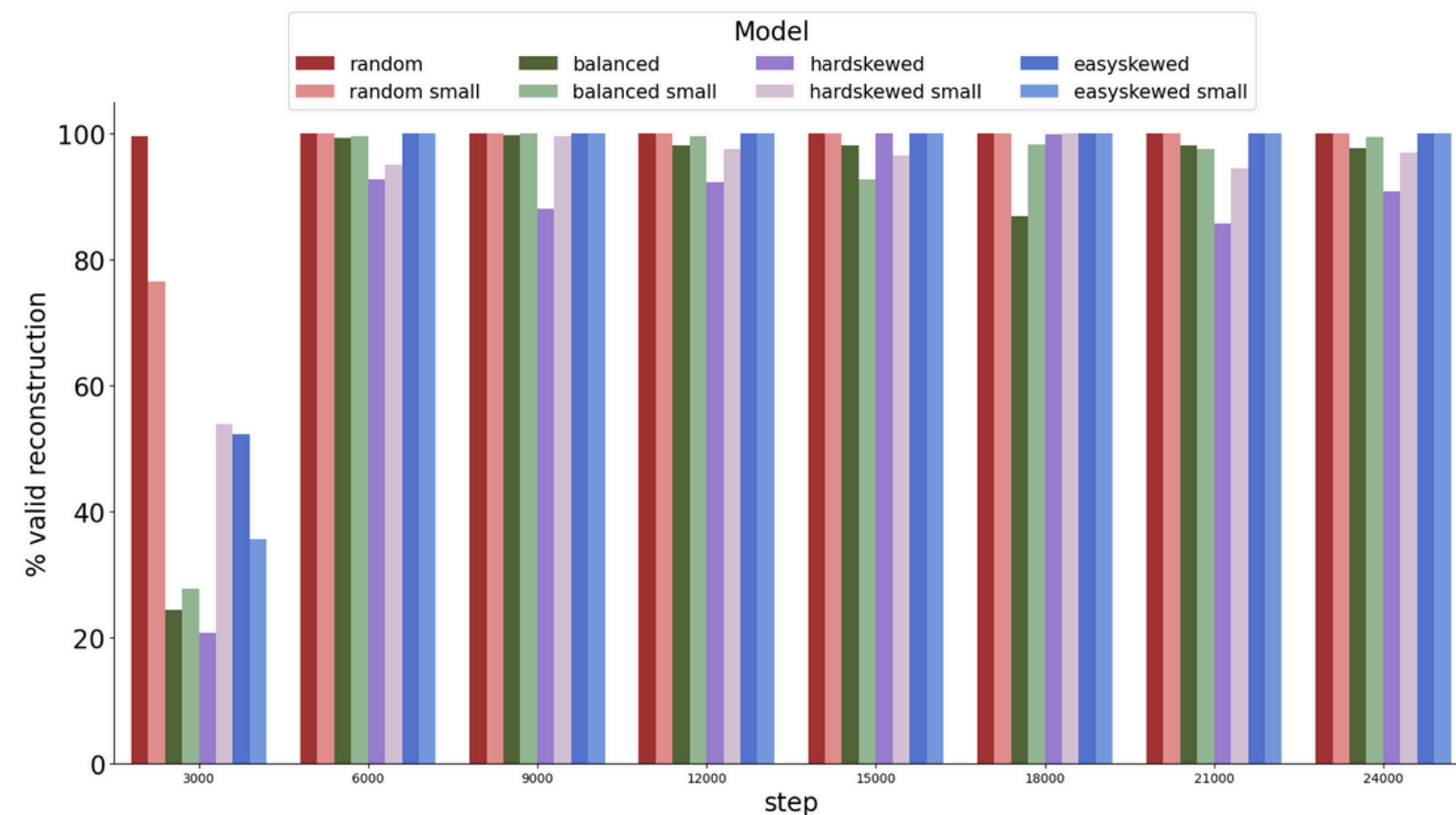
$\text{diff}(\mathbf{s}(\varphi), \mathbf{s}(\hat{\varphi}))$			
1quart	median	3quart	99perc
0.0225 (1)	0.0908 (4)	0.1797 (11)	0.4831 (47)
0.0159 (1)	0.0423 (2)	0.1354 (7)	0.7571 (85)
0.0224 (1)	0.0586 (3)	0.1631 (9)	0.7154 (81)
0.0073 (0)	0.0274 (1)	0.1085 (6)	0.6625 (75)
0.0106 (1)	0.0359 (2)	0.1159 (6)	0.5563 (61)

✓ syntax is learnt pretty soon



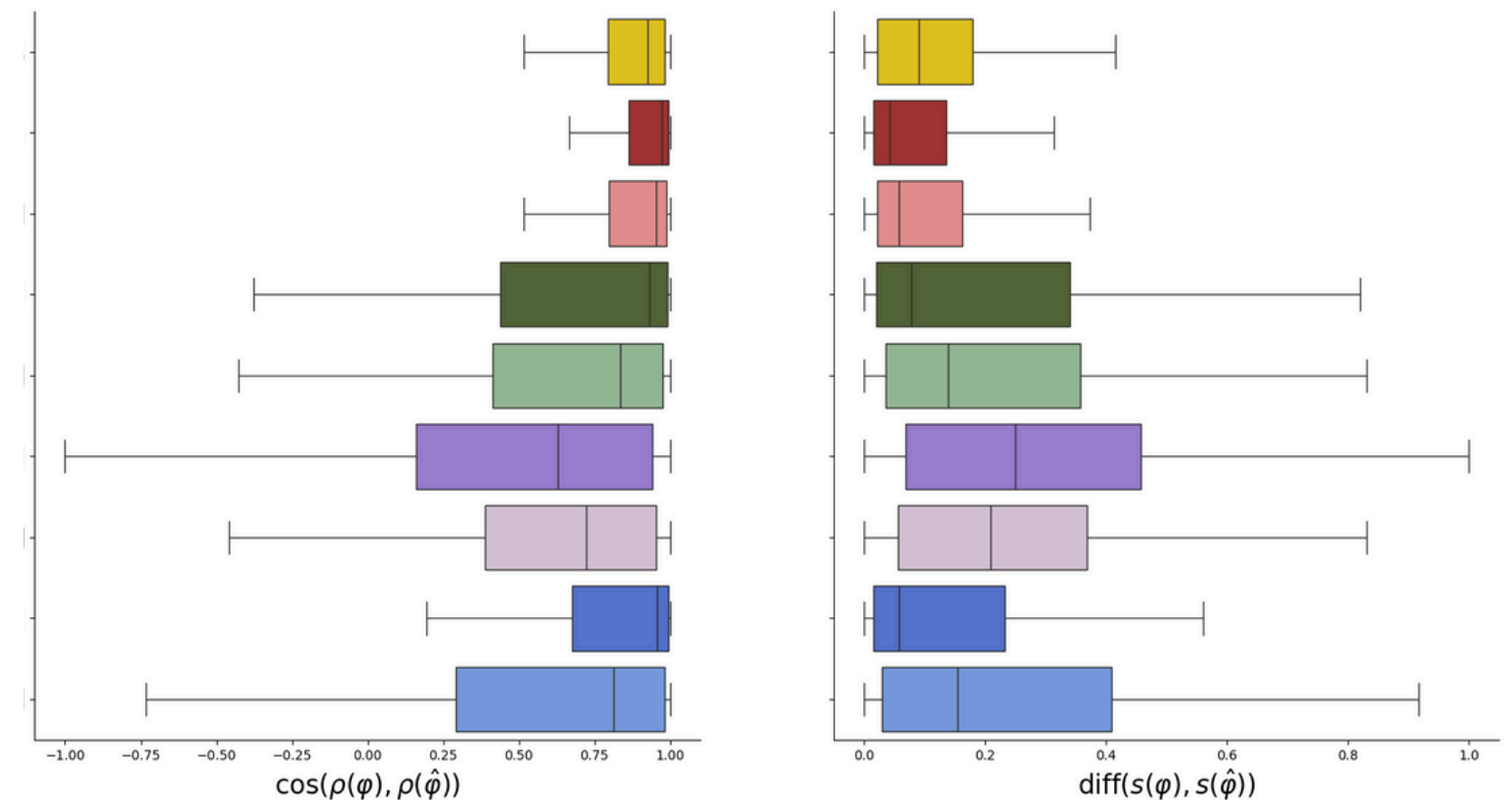
it suffices to train for 1 to 3 epochs to
learn valid formulae for all models

✓ syntax is learnt pretty soon



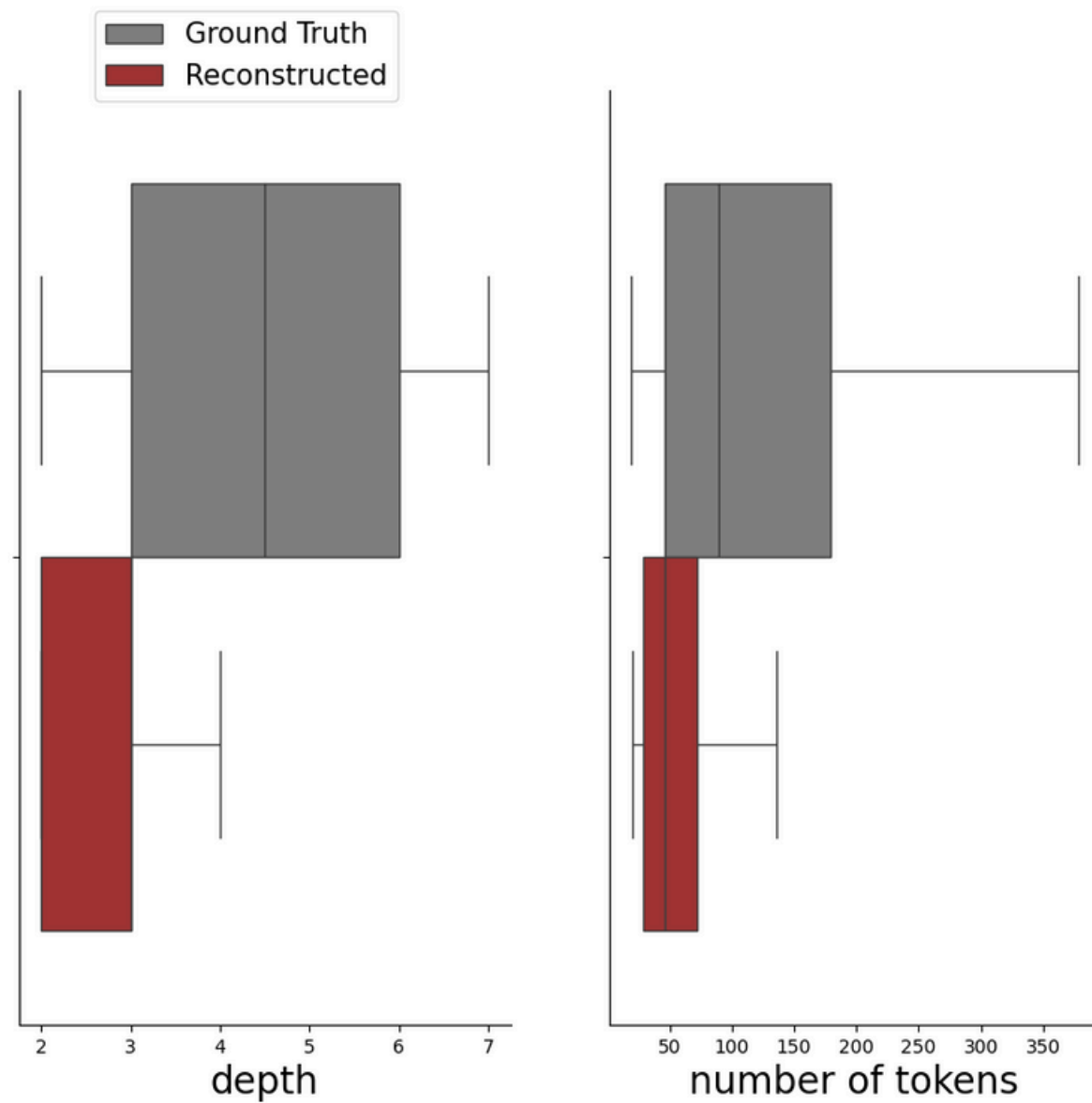
it suffices to train for 1 to 3 epochs to learn valid formulae for all models

✓ captures the semantics (better than the baseline)



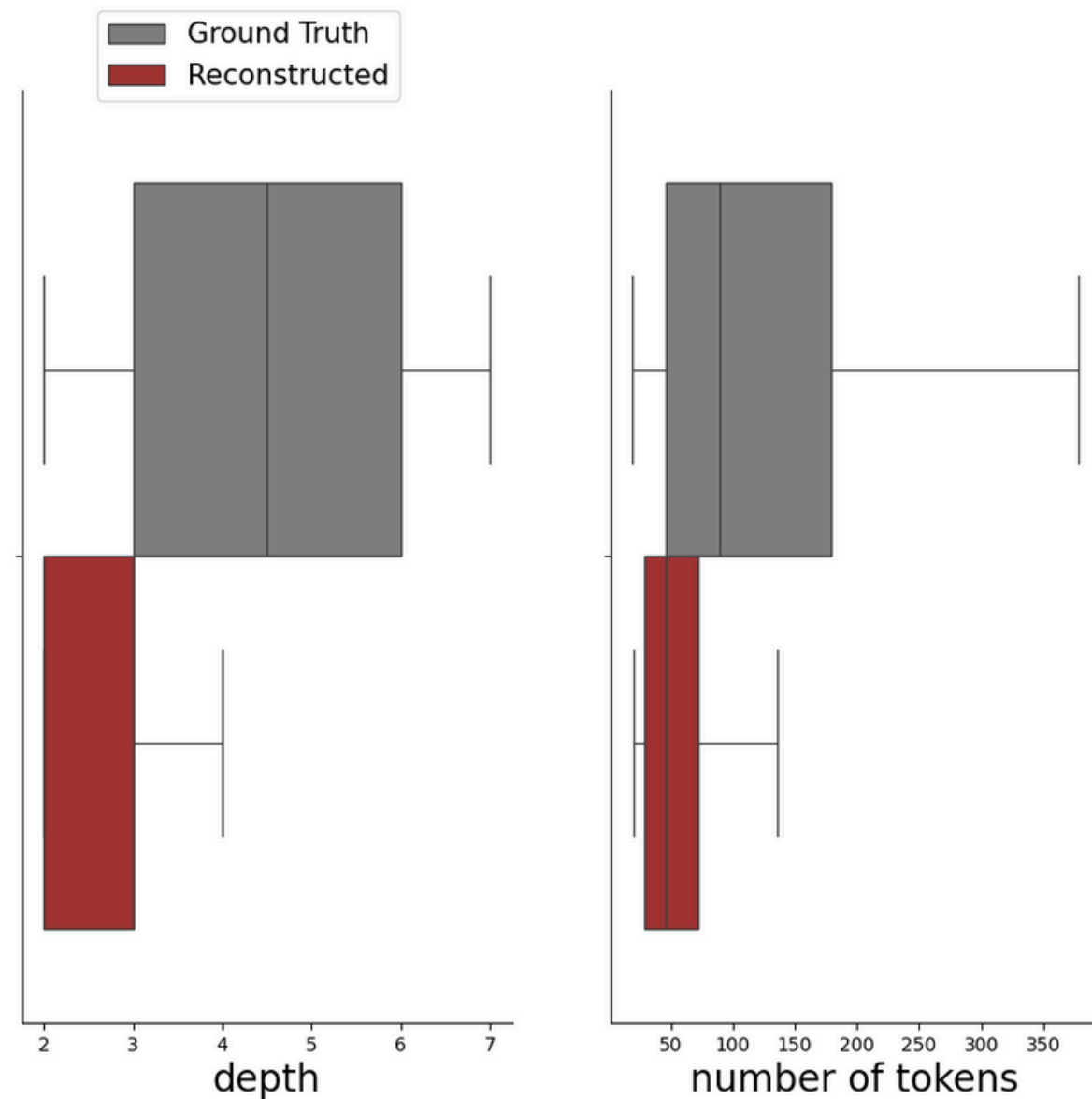
is able to outperform the vector DB on a (balanced) test set

✓ simpler decoded formulae



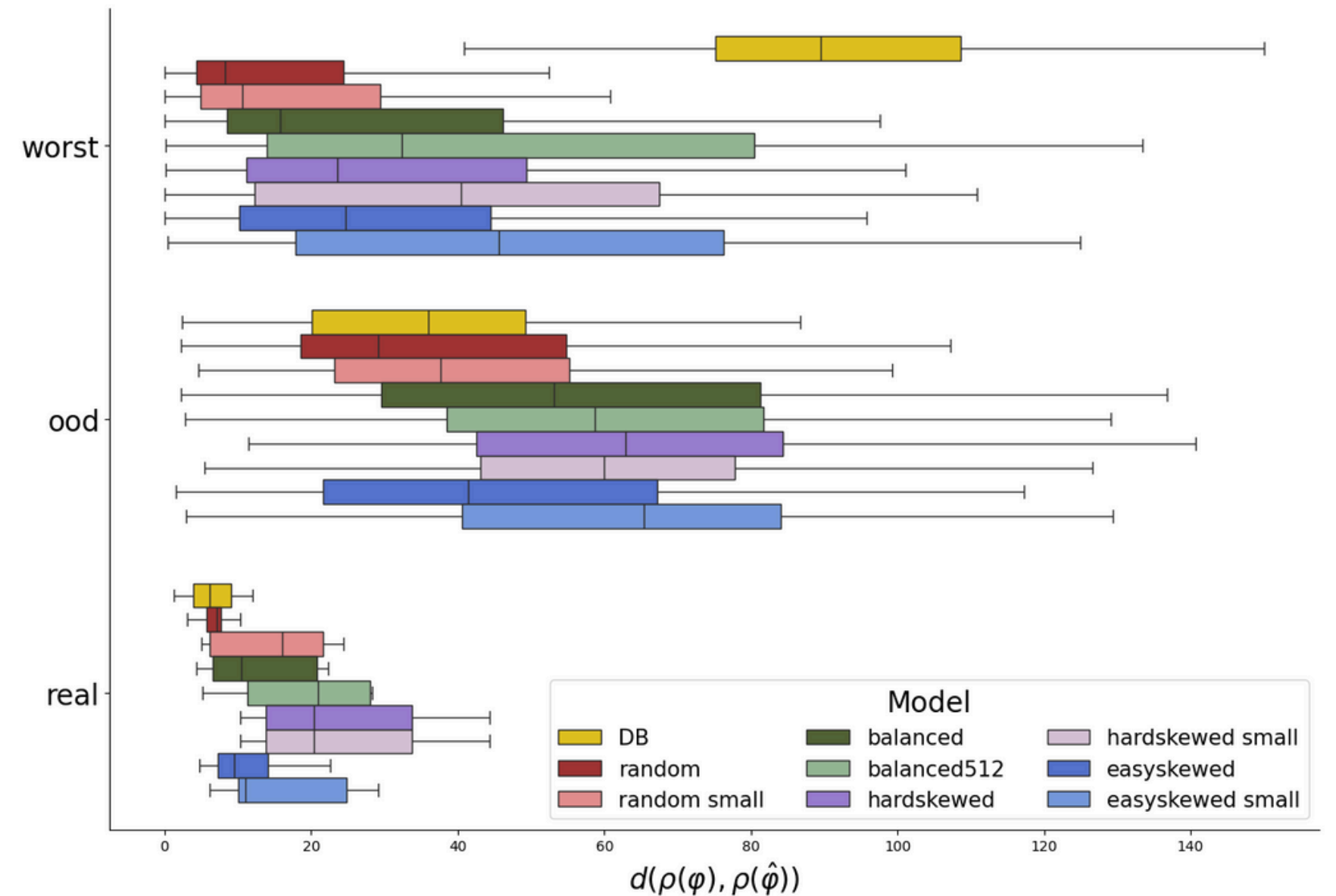
decoded formulae are **simpler**
than the ground-truth ones (i.e.
those of the starting embeddings)

✓ simpler decoded formulae



decoded formulae are **simpler** than the ground-truth ones (i.e. those of the starting embeddings)

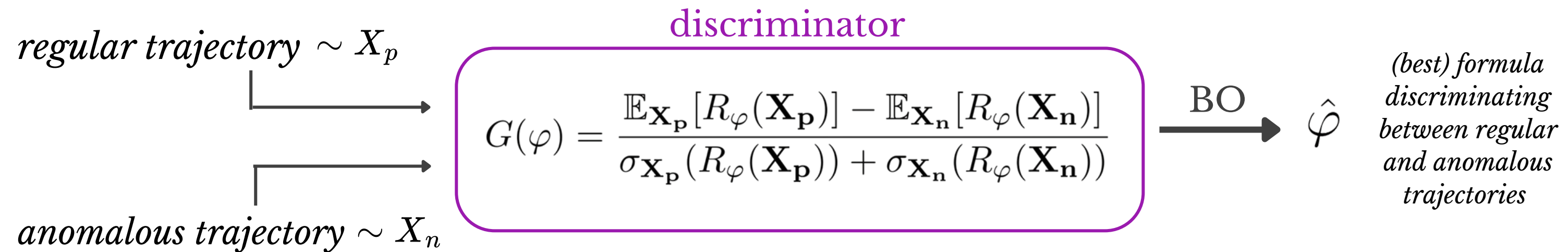
✓ robust approach



is robust OOD and on real-world CPS requirements

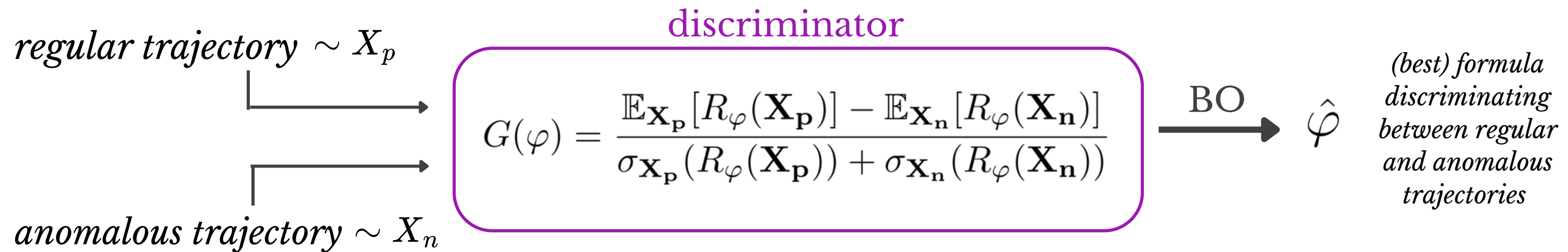
Requirement mining

🎯 automatically discover STL formulae that *distinguish between regular and anomalous behaviors of a system* by analyzing its trajectories



Requirement mining

- 🎯 automatically discover STL formulae that *distinguish between regular and anomalous behaviors of a system* by analyzing its trajectories



		$ \hat{\varphi}$	MCR	$Prec$
Linear	random (step 24K)	$G_{[3,\infty]}(x_0 \geq -11.44)$	0.0200 ± 0.0291	0.9650 ± 0.0489
	random small (step 24K)	$G(x_0 \geq -15.09)$	0.0150 ± 0.0300	0.9714 ± 0.0571
	random (step 48K)	$F(x_0 \geq 0.1918)$	0.0150 ± 0.0300	0.9700 ± 0.0600
	random small (step 48K)	$G(x_0 \geq -5.586)$	0.0150 ± 0.0201	0.9670 ± 0.0485

- 👉 the best mined formula is *highly discriminative, accurate and simple* (thus *interpretable*) in the test set across **3 different initialization seeds**

Conclusions and future directions

It is *possible to revert STL embeddings* into **valid** formulae with **equivalent semantics** to the ground truth, often resulting in **simpler** expressions in terms of length and nesting.

Conclusions and future directions

It is *possible to revert STL embeddings* into **valid** formulae with **equivalent semantics** to the ground truth, often resulting in **simpler** expressions in terms of length and nesting.

For 512-dim and 1024-dim models:

- ◆ need only **1 epoch** to produce a **valid** formula;
- ◆ **10 epochs** suffice to recover its **semantics**;
- ◆ are able to **simplify** a formula syntax;
- ◆ avoid the need of a vector database with a **fixed set of examples**.

Conclusions and future directions

It is *possible to revert STL embeddings* into **valid** formulae with **equivalent semantics** to the ground truth, often resulting in **simpler** expressions in terms of length and nesting.

For 512-dim and 1024-dim models:

- ◆ need only **1 epoch** to produce a **valid** formula;
 - ◆ **10 epochs** suffice to recover its **semantics**;
 - ◆ are able to **simplify** a formula syntax;
 - ◆ avoid the need of a vector database with a **fixed set of examples**.
-
- ✂ **Reduce** embedding dimensionality through PCA and UMAP: from 1024 to 16, 32, 64, 128 dimensions
 - 🔍 **Inspect** the embeddings to identify if there are **specific dimensions** that are **crucial** for accurate decoding.
 - 🌐 **Extend** this approach to **First-Order Logic** and **natural language**.

Thanks for the attention!



preprint



models & data



code

Get in touch:

✉ sara.candussio@phd.units.it

✂ @SaraCandussio

🦋 @saracandussio.bksy.social